


VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

# LAB ASSIGNMENT-02

**NAME : GILLEDI SANTHOSH**

**REGISTER NUMBER : 23MID0233**

**COURSE TITLE : ADVANCED PYTHON PROGRAMMING**

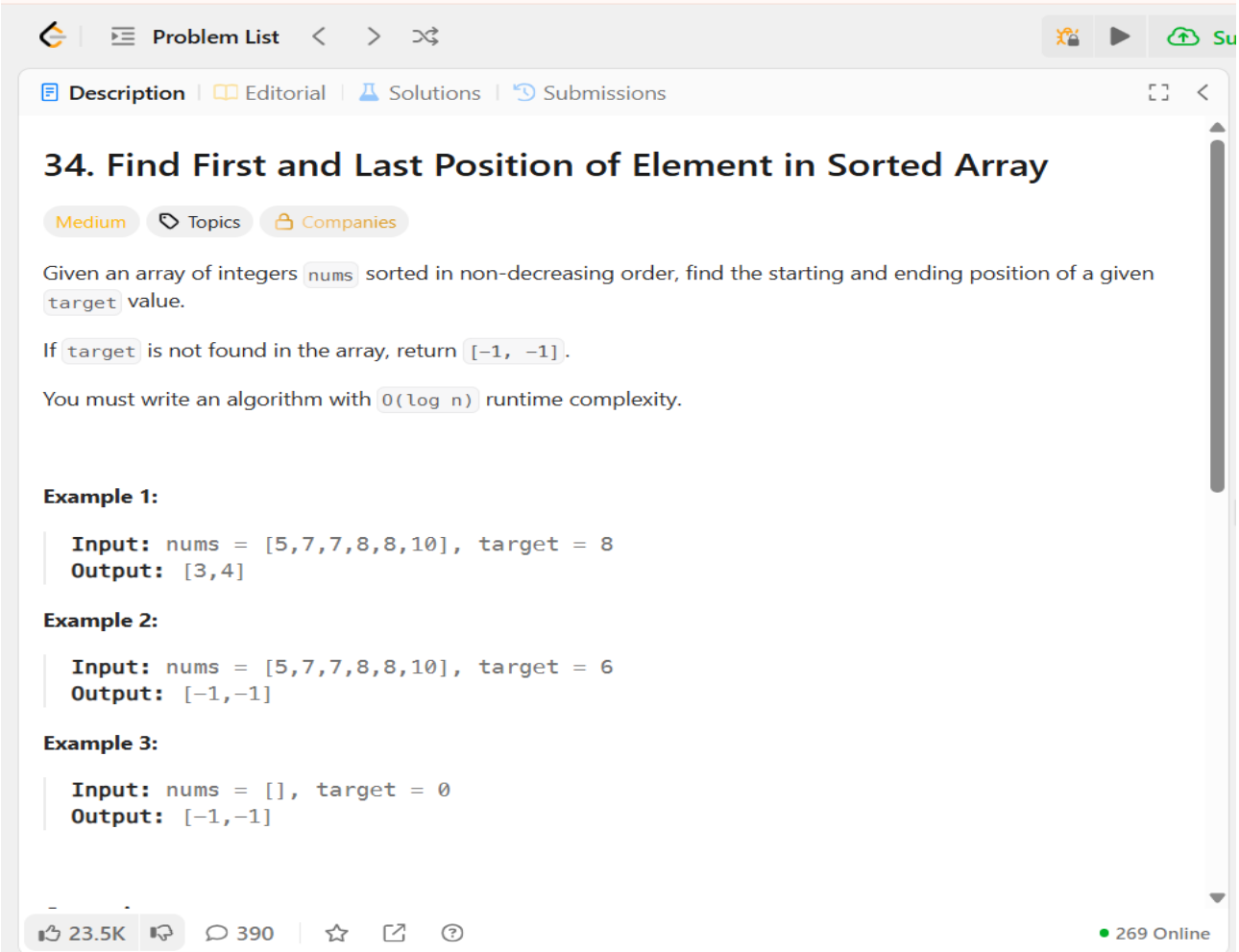
**COURSE CODE : CSI3007**

**COURSE FACULTY : SHARMILA BANU K**

**LAB SLOT : L31+L32+L49+L50**

**SUBMISSION DATE : 23-07-2026**

**Question 1:**



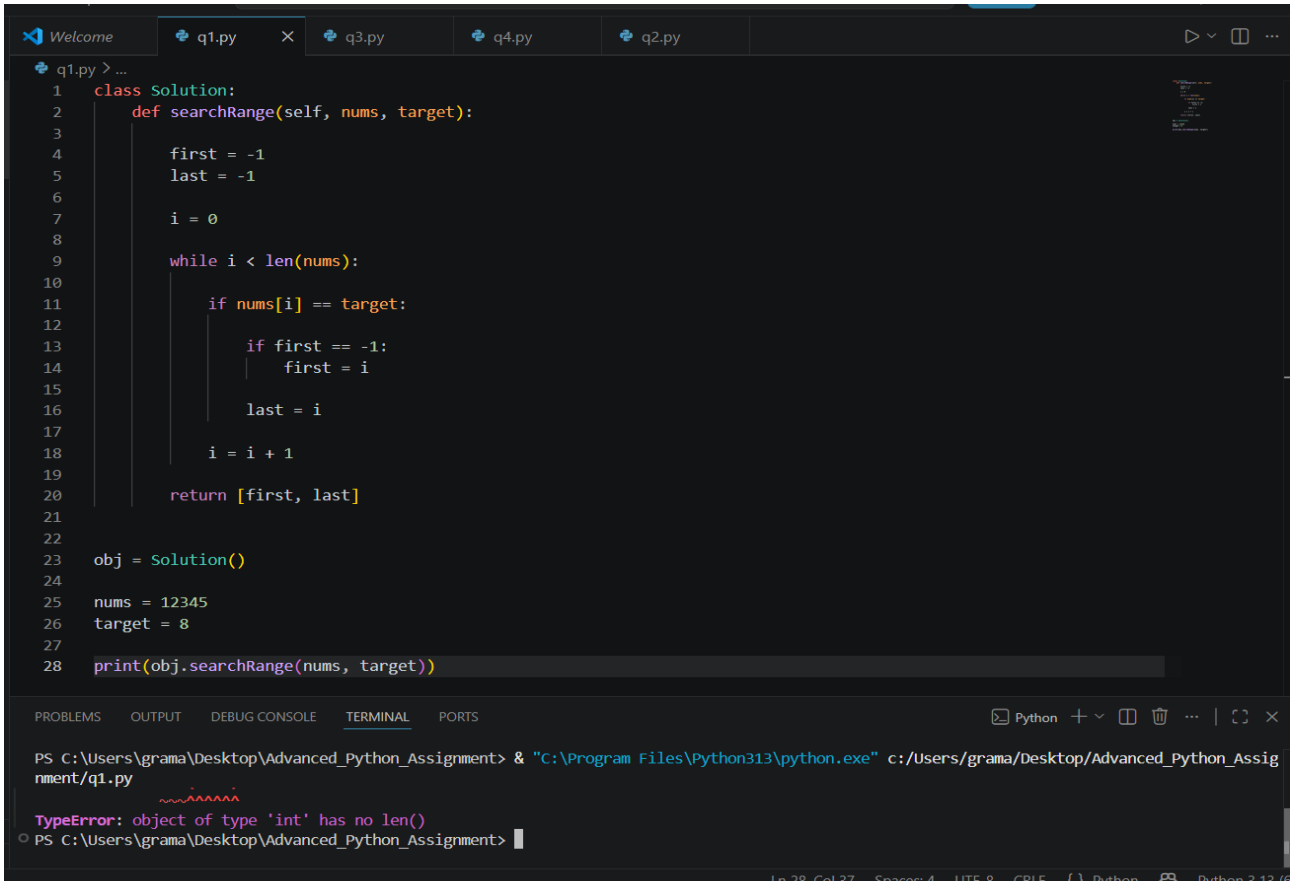

**Input:** List of integers (nums) and an integer (target).

nums = [5, 7, 7, 8, 8, 10]
target = 8

**Output:** List of two integers representing the first and last position of the target.

[3, 4]

## Error occurred:

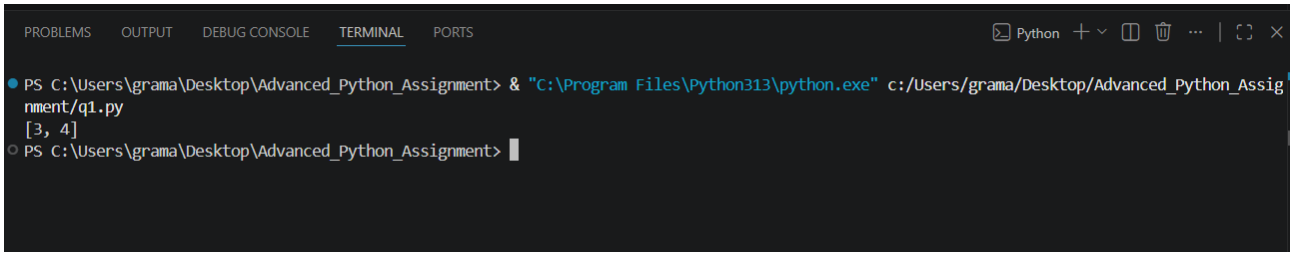



## Corrected one:

## Code:



**Output:**

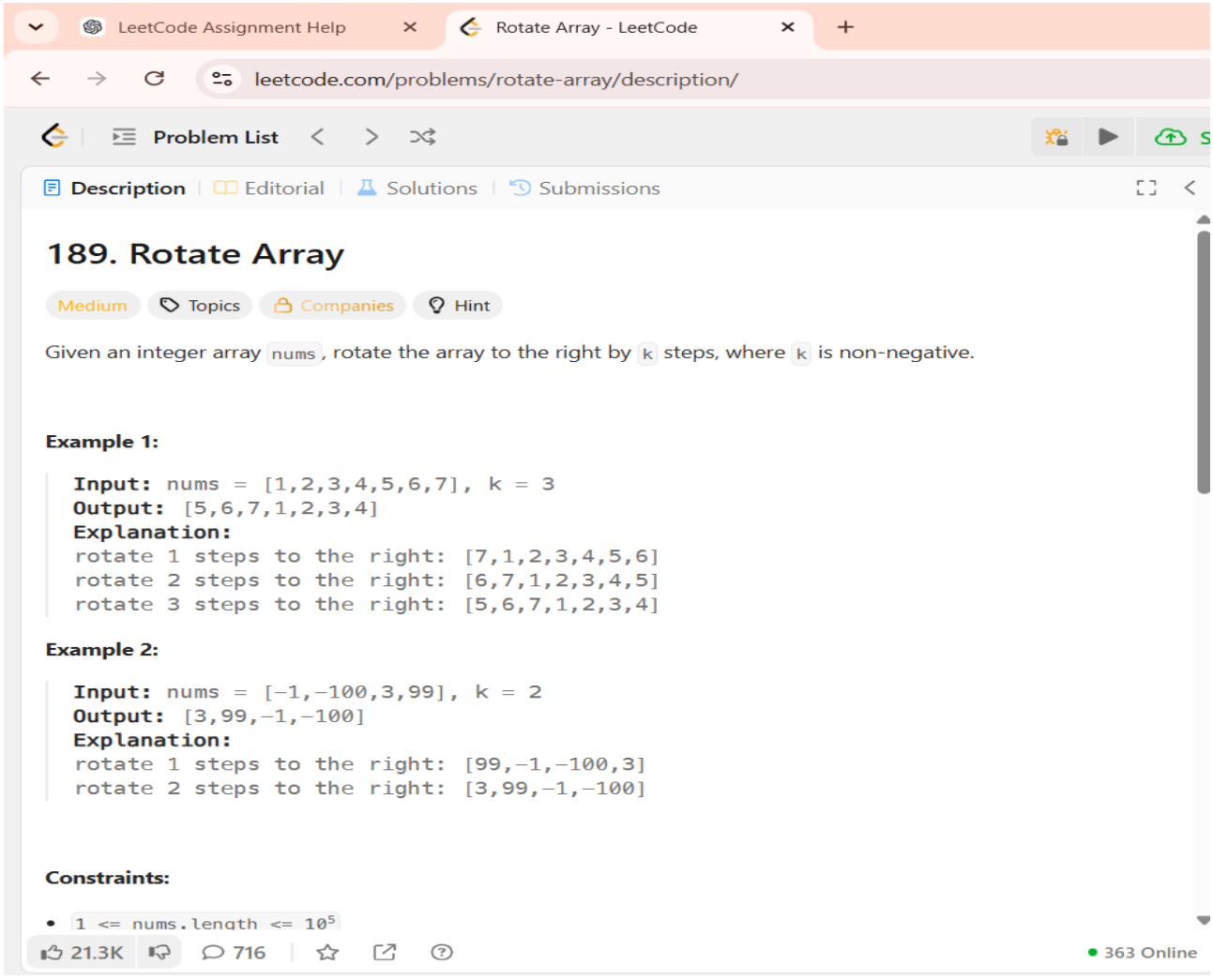



**Data Structure Used:** List

**Justification:**
I used a list because it can store multiple numbers together and makes it easy to check each element one by one.

**Logic:**
1.Start from the first element of the array.
2.Check each element with the target value.
3.If the target is found, store its first and last position.
4.Return the first and last positions of the target.

**Question 2:**



**Input:**
List of integers (nums) and an integer (k).

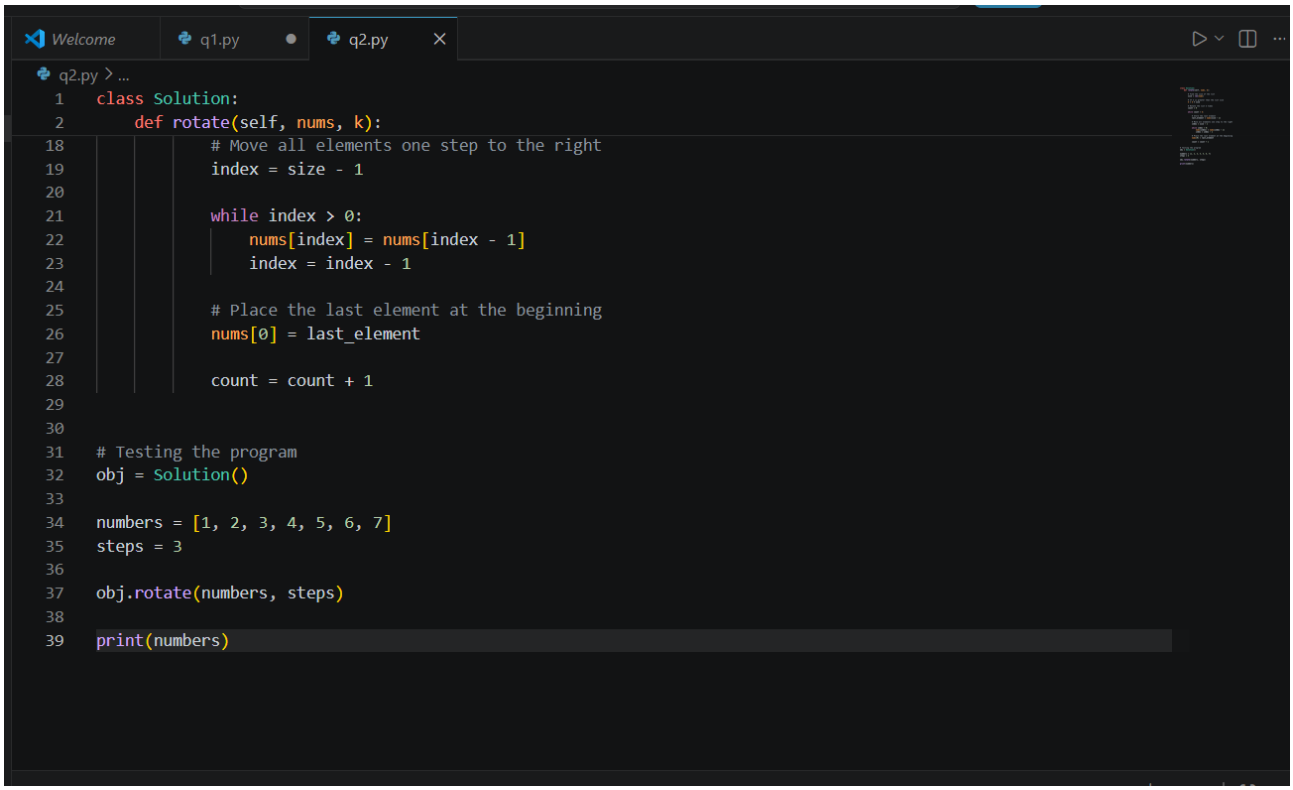
nums = [1, 2, 3, 4, 5, 6, 7]

k = 3

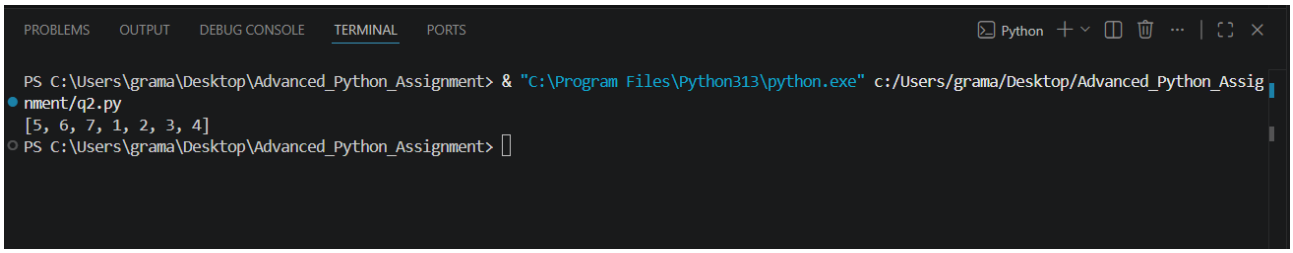
**Output:**
Rotated list of integers.

[5, 6, 7, 1, 2, 3, 4]

**Code:**




**Output:**



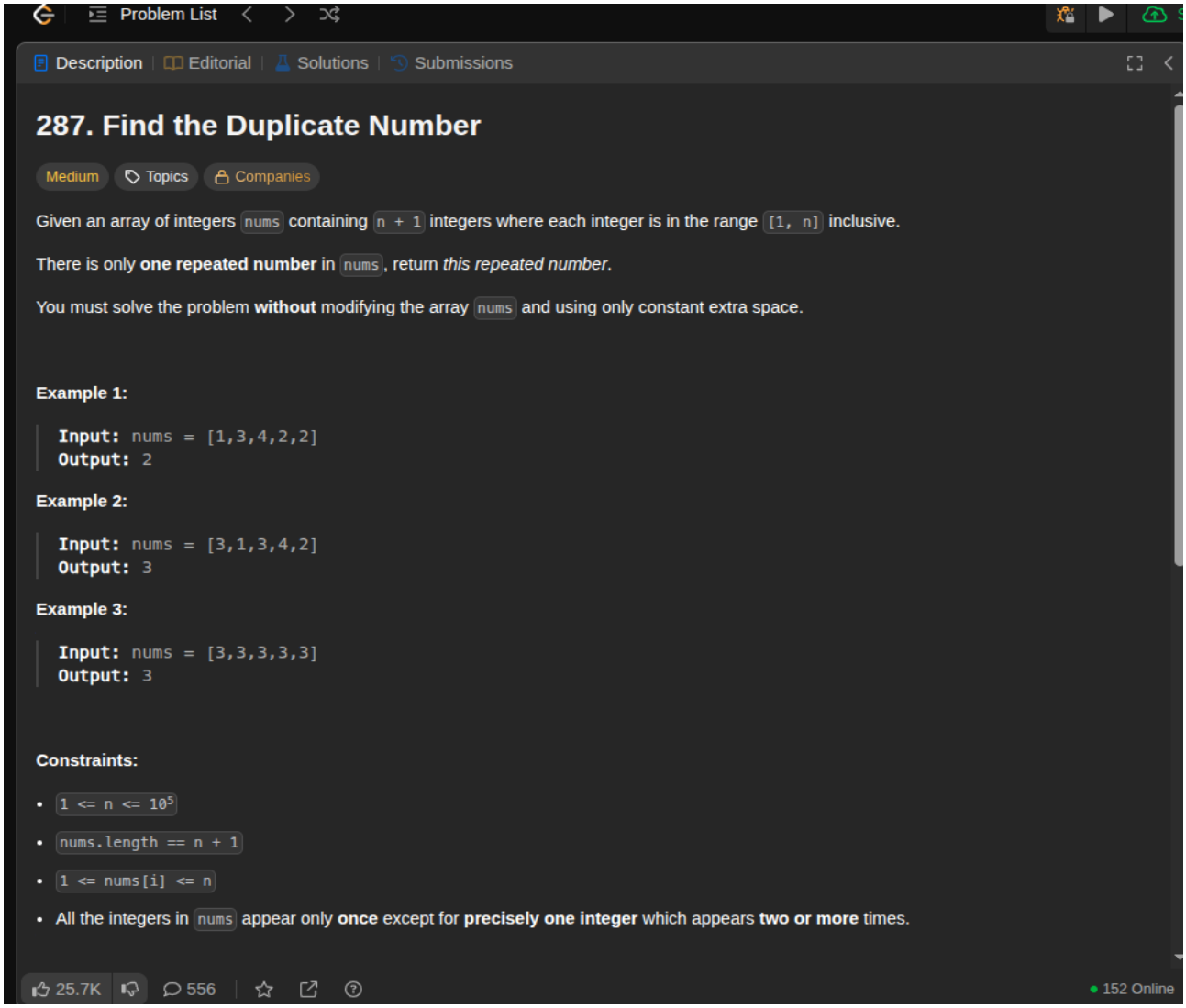

**Data Structure Used:** List

**Justification:**
I used a list because it stores all the numbers together. It is easy to access and change the elements while rotating the array.

**Logic:**
1.Find the size of the array.
2.Move the last element to the first position.

3.Shift the remaining elements to the right.
4.Repeat the process k times.

**Question 3:**



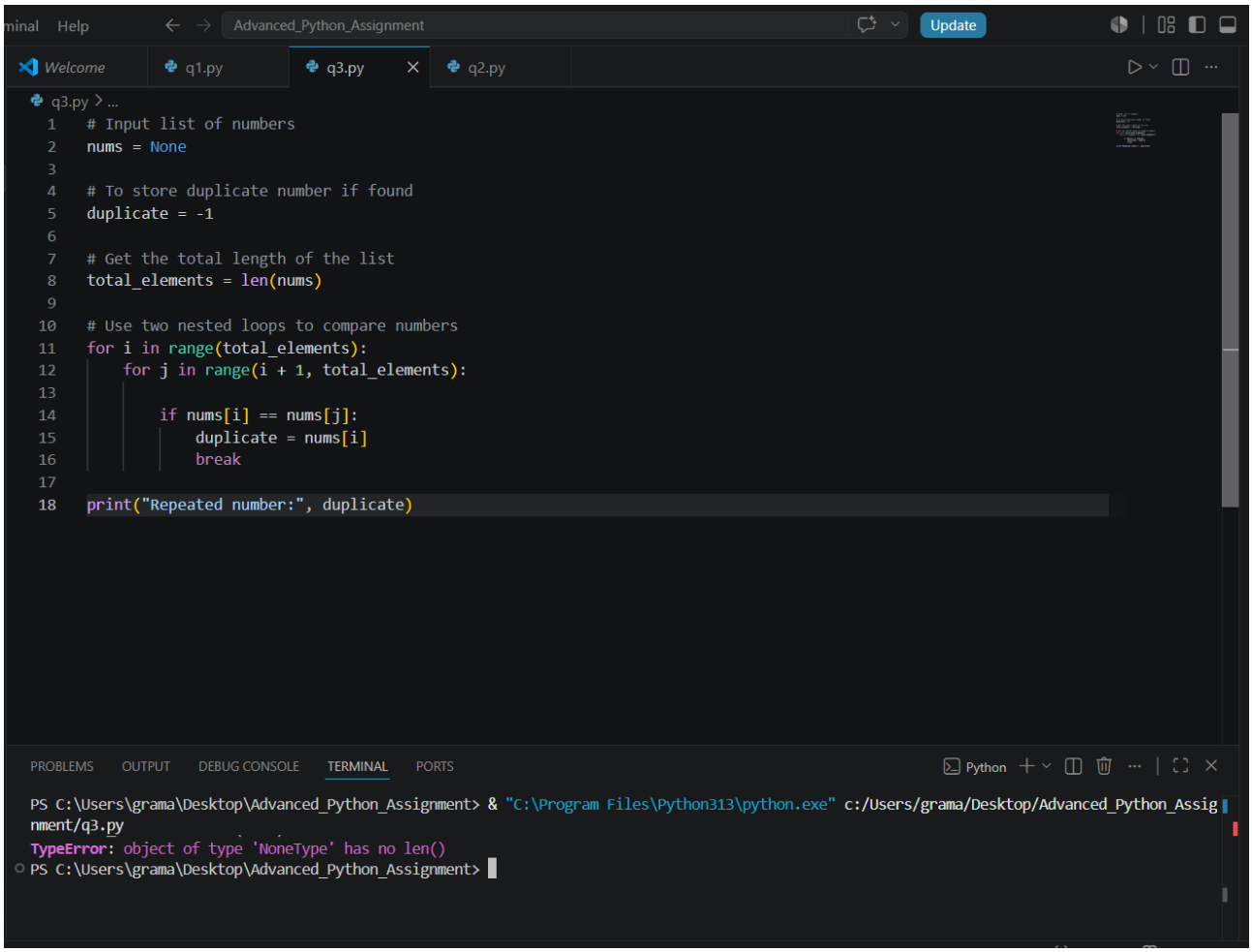

**Input:**
List of integers (nums).

nums = [1, 3, 4, 2, 2]

**Output:**
Integer (the duplicate number).

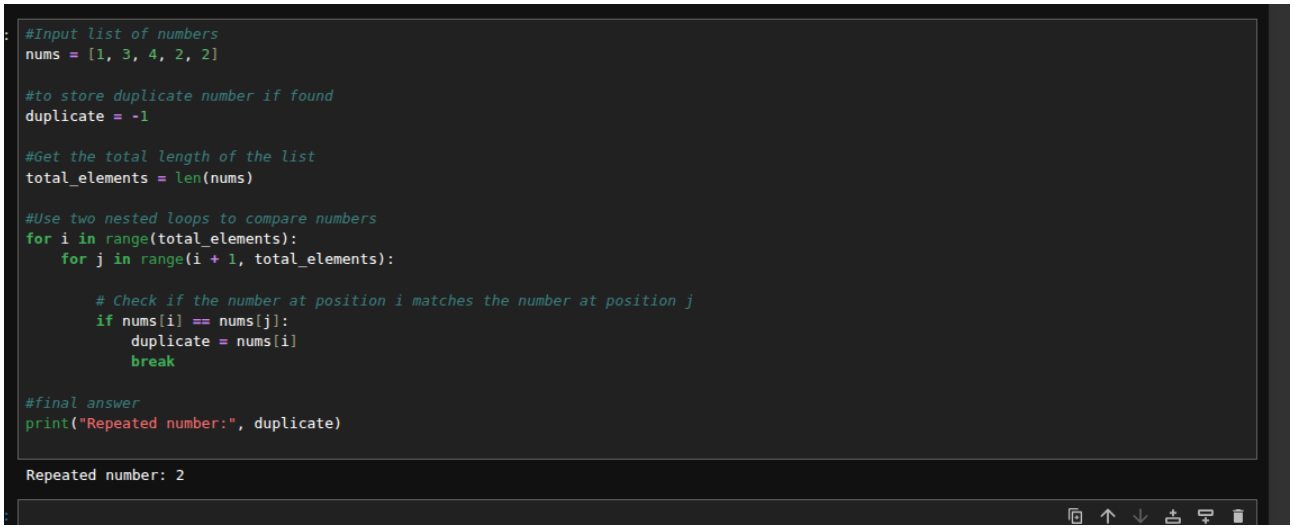
2

## Error occurred:

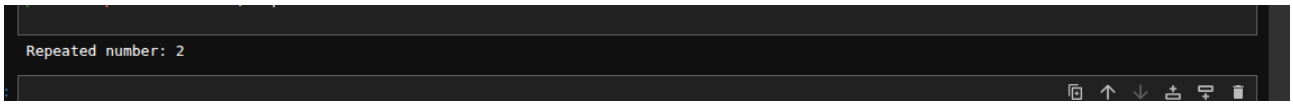



## Corrected one:

## Code:




## Output:

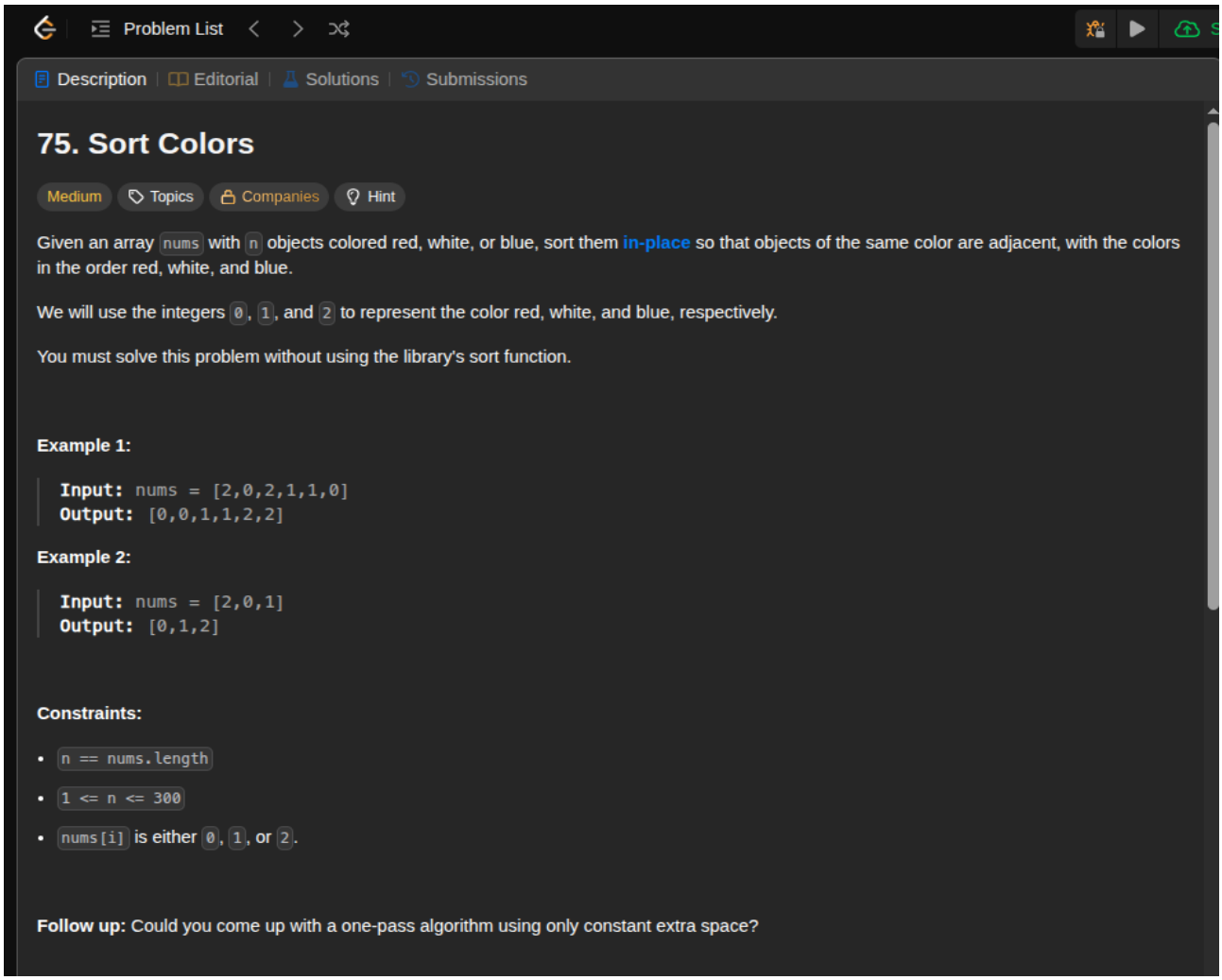


**Data Structure Used:** List

**Justification:**
I used a list because it stores all the numbers together. It is easy to compare each element with the remaining elements to find the duplicate number.

**Logic**
1.Store the numbers in a list.
2.Find the total number of elements in the list.
3.Compare each number with the remaining numbers in the list.
4.If two numbers are the same, store that number as the duplicate.
5.Display the duplicate number.

**Question 4:**



Problem List

Description | Editorial | Solutions | Submissions

## 75. Sort Colors

Medium | Topics | Companies | Hint

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

**Example 1:**

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

**Example 2:**

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

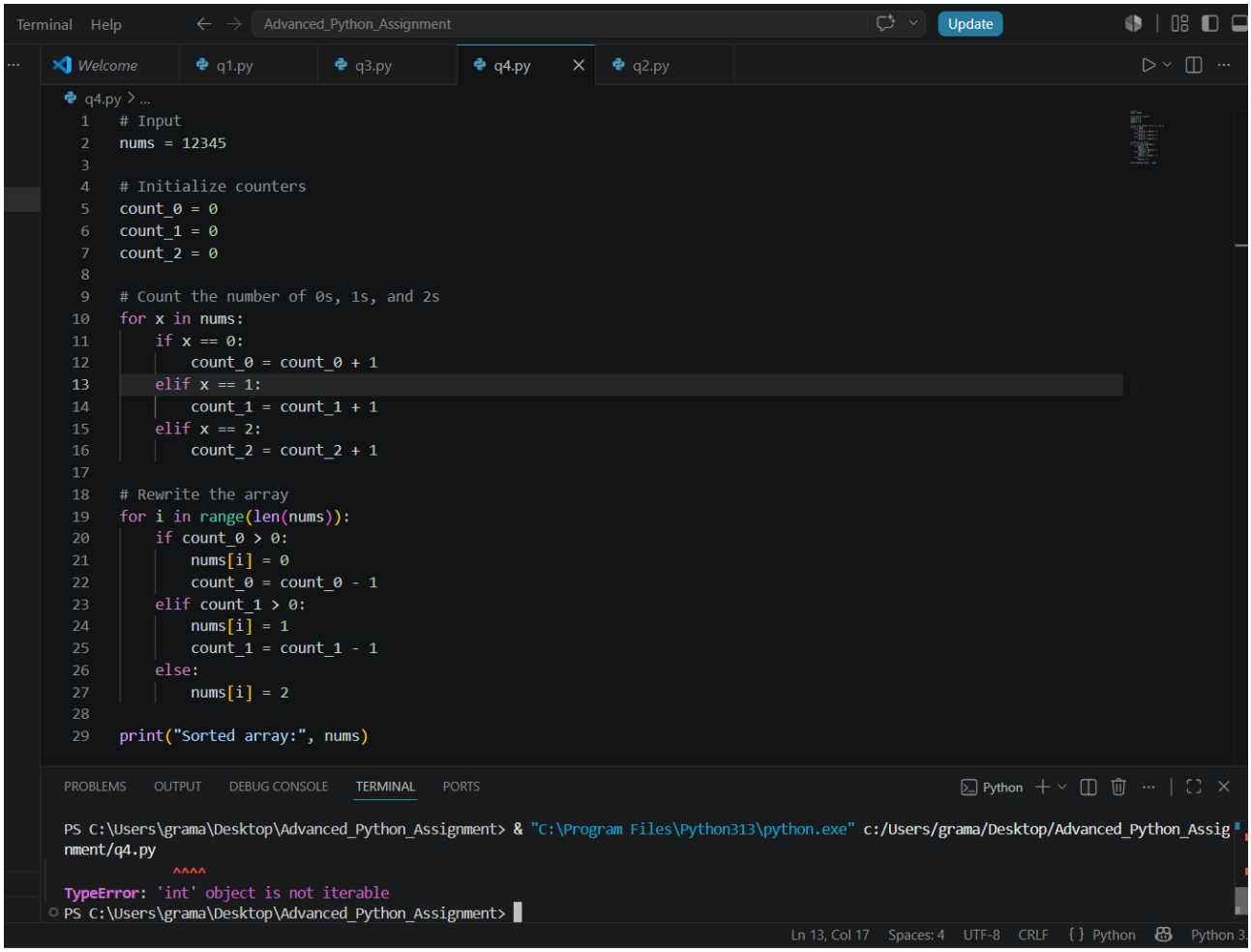
**Constraints:**

- `n == nums.length`
- `1 <= n <= 300`
- `nums[i]` is either `0`, `1`, or `2`.

**Follow up:** Could you come up with a one-pass algorithm using only constant extra space?

**Input:**
List of integers (nums) containing only 0, 1, and 2.
nums = [2, 0, 2, 1, 1, 0]
**Output:**
Sorted list of integers.
Sorted array: [0, 0, 1, 1, 2, 2]

**Error occurred:**




**Corrected one:**

**Code:**



**Output:**

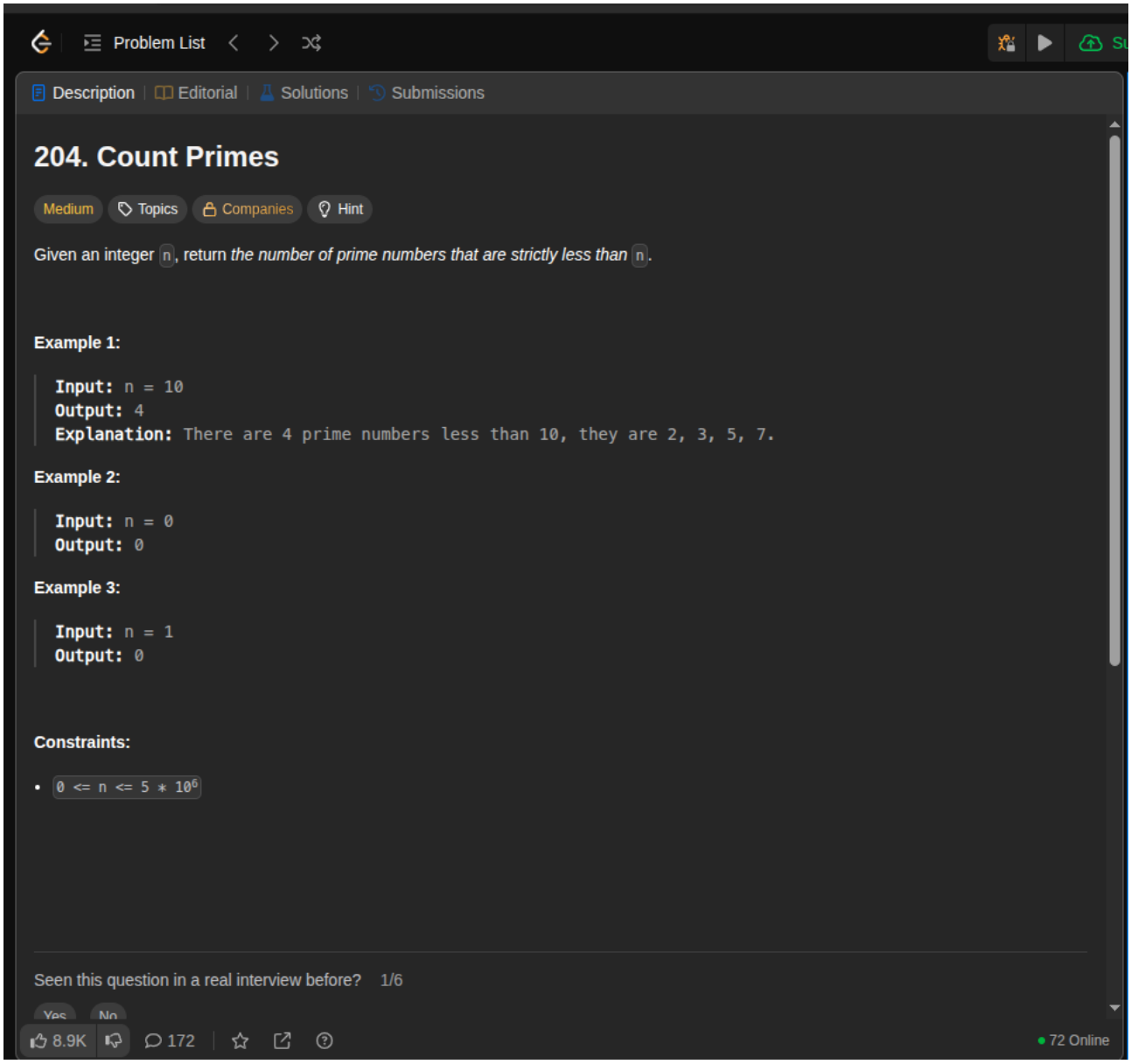



**Data Structure Used:** List

**Justification:**
I used a list because it stores all the numbers together. It is easy to count and arrange the elements in the required order.

**Logic:**
Count the number of 0s, 1s, and 2s in the array.
Replace the elements in the array using the counted values.
Fill all the 0s first, then 1s, and finally 2s.
Display the sorted array.

**Question 5:**



**Input:**
Integer (n).
n = 100
**Output:**
Integer (number of prime numbers less than n).
Number of primes: 25
**Output:**

```
[7]: #input
n = 100

prime_count = 0

#Loop from 2 up to n-1
for num in range(2, n):

    #Assume current number is prime
    is_prime = True

    #check any smaller number can divide the 'num' perfectly
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break

    #If no number divided it, 'is_prime' stays True, so we count it
    if is_prime == True:
        prime_count = prime_count + 1

#final total count
print("Number of primes:", prime_count)

Number of primes: 25

]:
```

**Data Structure Used:** Integer

**Justification:**
I used an integer because the program takes a single number as input and counts the number of prime numbers less than that number.

**Logic:**
Start checking numbers from 2 up to n - 1.
Check whether each number is divisible by any smaller number.
If it is not divisible, count it as a prime number.
Display the total number of prime numbers.